I Built a Mini SOC in VirtualBox and Attacked Myself (On Purpose)

So I've been working toward a SOC analyst role, and I kept hearing the same advice: *stop reading about detection and go build something that generates real alerts.* So I did. Two VMs, one Wazuh install, and an afternoon of attacking my own Ubuntu box from Kali. Here's the whole journey — including every single thing that went wrong, because honestly, that's the part that taught me the most.

The Setup

Nothing fancy — just VirtualBox on my Windows laptop:

- **Ubuntu VM** — the "server." This is where Wazuh lives (manager, indexer, dashboard, all on one box).
- **Kali VM** — the "attacker." Nmap, Hydra, and an SSH client.
- Both talking to each other over a VirtualBox host-only network, `192.168.56.0/24`.

The plan: get the two VMs talking to each other, install Wazuh on Ubuntu, then throw some basic attacks at it from Kali and watch the alerts roll in.

Step 1: Just Getting the VMs to Ping Each Other

You'd think this would be the easy part. It mostly was, except Kali's `eth1` interface didn't have an IP address at all when I first checked — `ip a show eth1` came back basically empty. Turned out I needed to manually assign one:

```
sudo ip addr add 192.168.56.102/24 dev eth1
```

That worked, but it was **temporary** — gone on reboot. Fixed it properly with NetworkManager:

```
sudo nmcli con add type ethernet ifname eth1 con-name hostonly ipv4.method auto
sudo nmcli con up hostonly
```

Once both VMs had addresses, `ping` between them worked cleanly, 0% packet loss both ways. Small win, but it felt good.

Step 2: The SSH Install That Wouldn't Install

Tried to install OpenSSH server on Ubuntu so Kali would have something to attack. Got hit with this:

```
404 Not Found [IP: 91.189.92.24 80]
```

Turns out my package index was just stale — the exact version the system remembered didn't exist on the mirror anymore. The fix was almost embarrassingly simple:

```
sudo apt update
sudo apt install openssh-server
```

Lesson learned: when apt throws a 404 on a specific package version, it's usually not your fault — the index is just out of date. Refresh first, panic later.

just out of date. Refresh first, panic later.

Step 3: "Is SSH Even Running?"

After installing, `systemctl status ssh` showed `inactive (dead)`, which made my stomach drop for a second. Turns out Ubuntu 24.04 uses **socket activation** — `ssh.socket` listens on port 22 and only spins up `ssh.service` when a connection actually comes in. So "dead" didn't mean broken, it meant *idle*. A quick `nc -zv` from Kali confirmed the port was genuinely open. If you want it always-on instead of on-demand, `systemctl enable --now ssh` does that.

Step 4: Running Out of Disk Space Before Even Installing Wazuh

Wazuh's all-in-one install is genuinely heavy — indexer, manager, dashboard, all on one box. My Ubuntu VM only had 25 GB allocated, which felt tight. Resizing a live VM's disk without blowing anything away turned out to be a two-step process:

Step A — resize the virtual disk file (host side, VM powered off):

```
VBoxManage modifymedium disk "C:\path\to\ubuntuvm2.vdi" --resize 35840
```

Step B — grow the actual filesystem inside Ubuntu (guest side):

```
sudo growpart /dev/sda 2
sudo resize2fs /dev/sda2
```

I almost ran `growpart` against the wrong partition (`sda1` is a tiny 1 MB boot partition, not the one you want) — always double-check with `lsblk` first. Worked perfectly once I pointed it at `sda2`, the actual 25 GB root partition, and I ended up with a comfortable 35 GB with zero data loss.

Also — fun fact — `VBoxManage` wasn't even in my Windows PATH, so every command had to start with the full path:

```
"C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" ...
```

until I fixed that with `setx`.

Step 5: Installing Wazuh (The Easy Part, Finally)

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
```

About 10–15 minutes later, it spat out an admin username and a password I immediately copied somewhere safe. Dashboard URL: `https://192.168.56.101`.

Step 6: "No Agents Registered" — Wait, Did It Even Work?

The Overview page greeted me with: "*This instance has no agents registered.*" My first instinct was to install the standalone `wazuh-agent` package to monitor the server itself. That backfired immediately:

```
dpkg: wazuh-agent conflicts with wazuh-manager
wazuh-manager (version 4.14.8-1) is present and installed.
```

Turns out — and this took some digging — the `wazuh-manager` package already *includes* its own log collection for the host it's installed on. Installing `wazuh-agent` on the same box is not just unnecessary, it's actively blocked. The "no agents registered" message only refers to *remote* agents monitoring other machines. The manager watches itself automatically through its own `localfile` config, including a `journald` collector that catches basically everything — SSH, sudo, PAM, the works.

I confirmed this by tailing the alert log directly:

```
sudo tail -f /var/ossec/logs/alerts/alerts.log
```

and just... watching alerts appear in real time as I did normal things like running `sudo`. That was oddly satisfying.

Step 7: Kali Suddenly Couldn't Reach the Internet

Mid-lab, I tried `apt update` on Kali and got `Temporary failure resolving 'http.kali.org'`. Checked `ip route` and — nothing. No `default via` line at all. Kali had a working host-only connection to Ubuntu but literally no route out to the internet. A NAT adapter hiccup, apparently, because after some interface poking it sorted itself out and `ping 8.8.8.8` started working again. Networking gremlins — sometimes the fix really is just "check again in a bit."

Step 8: Finally, the Actual Attack

With everything working, here's the fun part. From Kali:

```
ssh fakeuser@192.168.56.101
```

Wrong password a few times, and almost instantly, the Ubuntu alert log lit up:

```
Rule: 5710 (level 5) -> 'sshd: Attempt to login using a non-existent user'
Src IP: 192.168.56.102
```

Then the real test — a Hydra brute-force with a tiny wordlist:

```
hydra -l fakeuser -P /tmp/passlist.txt -t 4 ssh://192.168.56.101
```

Same rule fired, five times in a row, clustered within seconds — exactly the pattern that distinguishes "someone brute-forcing me" from "I fat-fingered my password once." The dashboard's alert counters ticked up live as it ran.

And for contrast, I logged in with the *real* password:

```
ssh ubuntuvm2@192.168.56.101
```

Different rule entirely — **5715, level 3,** `authentication_success` — way lower severity than the failed attempts. That's actually a neat detail: Wazuh treats failed logins against unknown usernames as more suspicious than successful ones, by design, even before you factor in volume or speed.

What I Actually Learned

Technically: how Wazuh's local log collection works without a separate agent, how VirtualBox networking actually routes traffic, and how to read rule severity logic instead of just alert counts.

But honestly, the bigger lesson was about *troubleshooting itself*. Nearly every step had a hiccup — stale packages, disk space, missing routes, a package conflict I didn't see coming — and every single one was solvable by reading the actual error message instead of guessing. That's a very SOC-analyst skill in itself: don't panic, read the output, fix the thing it's actually telling you is broken.

Next up: adding a second machine as a real remote agent, and maybe writing a custom rule to auto-flag anything that looks like this Hydra pattern again.